EchoSign: An Integrated AI Platform for Real-Time American Sign Language Translation, Gesture Evaluation and Speech Synthesis

Report submitted in partial fulfillment of the requirement for the degree of

B. Tech.

In

Computer Science and Engineering

Under the Supervision of

Ms. Rita Nagar

Assistant Professor, C.S.E.

Submitted By:

Nidhi Singh (2303491530068)

Yuvraj Singh (2303491530128)

Rohit Kumar (2303491530090)

Prashant Kumar Singh (2303491530075)

Komal Goswami (2303491530061)

Department of Computer Science & Engineering

Maharana Pratap Engineering College, Kanpur

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Session: 2026-27

DECLARATION

This is to certify that the Synopsis Report entitled “EchoSign: An AI-Powered American Sign Language Translation System Using Deep Convolutional Neural Networks and Real-Time Speech Synthesis”, which is submitted in partial fulfilment of the requirement for the award of the degree of B.Tech. in Computer Science and Engineering to MPEC Kanpur, Dr. A.P.J. Abdul Kalam Technical University, Lucknow, comprises only original work and studies carried out by the students themselves. The matter embodied in this report has not been submitted for the award of any other degree.

We further declare that all source code, database architectures, algorithmic modules, and technical documentation presented in this report have been implemented in accordance with academic integrity guidelines. All external open-source libraries and scholarly references utilized have been duly credited and cited.

Date: ……

Place: Kanpur

Nidhi Singh (2303491530068)
Yuvraj Singh (2303491530128)
Rohit Kumar (2303491530090)
Prashant Kumar Singh (2303491530075)
Komal Goswami (2303491530061)

Approved By:

Ms. Rita Nagar
Project Guide
CSE Department
MPEC, Kanpur
Signature: …….

Mr. Milind Bhatt
HOD
CSE Department
MPEC, Kanpur
Signature: …….

TABLE OF CONTENTS

Title Page ..... **i**
Declaration ..... **ii**
Table of Contents ..... **iii**
Abstract ..... **iv**
List of Acronyms, Figures and Tables ..... **v**
Chapter 1 – INTRODUCTION **1**
1.1 Fundamentals of Web-Based Sign Language Translation Platforms 1
1.2 Background Theory 1
1.3 Core Principles 2
1.4 Problems in the Current System 2
Chapter 2 – LITERATURE SURVEY **2**
2.1 Overview of Existing Systems 2
2.2 Literature Review 2
2.2.1 Technique 1: Wearable Sensor-Instrumented Data Gloves 2
2.2.2 Technique 2: Isolated Classical Computer Vision Contours 2
2.2.3 Technique 3: Coordinate Landmark Regressors (MediaPipe / OpenPose) 3
2.2.4 Technique 4: Deep Convolutional Neural Networks (CNNs) 3
2.3 Summary 3
Chapter 3 – INFERENCES FROM LITERATURE **3**
3.1 Inferences Drawn from Literature Survey 3
Chapter 4 – PROBLEM STATEMENT AND SOLUTION APPROACH **4**
Problem Statement and Solution Approach **4**
4.1 Proposed Work 4
4.2 Methodology 5
4.3 Performance Metrics & System Verification 5
Chapter 5 – SYSTEM ARCHITECTURE & IMPLEMENTATION DETAILS **6**
5.1 Data Flow Diagrams & Subsystem Architecture 6
5.2 Database Schemas & REST API Specifications 7
5.3 Deep CNN Architecture & Speech Synthesis Implementation 8
5.4 Software Testing Matrix & Execution Verification 9
Conclusion **10**
REFERENCES **10**

ABSTRACT

Individuals who are deaf or hard-of-hearing frequently encounter severe communication barriers in public, healthcare, and educational environments due to the widespread absence of sign language literacy among the hearing population. While certified human sign language interpreters provide effective translation, their availability is severely limited and cost-prohibitive for everyday social and commercial interactions. Conventional automated assistive systems either depend on cumbersome, sensor-instrumented gloves or high-parameter deep learning models that require specialized desktop GPUs, rendering them unviable for spontaneous mobile use.

This project presents EchoSign, an end-to-end, full-stack web application architected to deliver real-time American Sign Language (ASL) fingerspelling recognition, word composition, and spoken English audio synthesis within a lightweight browser interface. Developed using Flask, TensorFlow Lite, and Bootstrap 5, EchoSign classifies hand gestures across 29 discrete classes—including the 26 standard alphabet letters (A–Z) and 3 operational control gestures (space, delete, and neutral resting state).

The system incorporates a deep Convolutional Neural Network (CNN) trained on a curated corpus of 20,300 color images (700 images per class) derived from the benchmark ASL Alphabet dataset. To address the common domain gap between training and web deployment, EchoSign enforces exact preprocessing parity: both data ingestion and real-time inference employ a center-square aspect-ratio crop and Pillow bilinear resampling down to a uniform 64×64 pixel grid.

The model parameters undergo float16 quantization into TensorFlow Lite format, reducing storage from 26.99 MB to 4.29 MB and enabling serverless execution on standard CPU runtimes. Comprehensive validation across 2,030 held-out test images demonstrates a 96.75% classification accuracy and an average serverless inference latency under 25 ms. An integrated HTML5 Region-of-Interest (ROI) canvas and the browser-native Web Speech API enable seamless text vocalization without external API costs.

Keywords: *American Sign Language, Convolutional Neural Networks, Edge Inference, TensorFlow Lite, Bilinear Interpolation, Web Speech API, Assistive Technology.*

LIST OF ACRONYMS AND ABBREVIATIONS

Acronym	Expansion	Acronym	Expansion
API	Application Programming Interface	O(1)	Constant-time complexity
ASL	American Sign Language	TFLite	TensorFlow Lite Engine
CNN	Convolutional Neural Network	PIL	Python Imaging Library (Pillow)
REST	Representational State Transfer	ROC-AUC	Receiver Operating Characteristic AUC
JSON	JavaScript Object Notation	SLA	Service Level Agreement (< 50 ms)

LIST OF FIGURES

- Fig. 1.1: End-to-End Three-Tier Architecture of EchoSign Platform
- Fig. 1.2: Confusion Matrix across 29 ASL Classes (Held-Out Test Set)
- Fig. 1.3: Training Convergence & Loss Minimization Curves across 14 Epochs
- Fig. 1.4: Bilinear Interpolation Resampling Grid and Fractional Offsets
- Fig. 1.5: Level 1 Data Flow Diagram (DFD) of EchoSign Subsystems
- Fig. 1.6: ROI Canvas Frame Ingestion and Client Debounce Workflow

LIST OF TABLES

- Table 2.1: Comparative Summary of Existing Gesture Portals vs. EchoSign
- Table 4.1: Performance across Candidate Classification Thresholds
- Table 4.2: Latency Accounting across Pipeline Stages vs the 50 ms SLA
- Table 5.1: Pipeline Preprocessing Schemas, Types and Constraints
- Table 5.2: RESTful API Route Specifications and Contract Definitions
- Table 5.3: Automated Test Suite Verification Results (8/8 Passed)

CHAPTER 1
INTRODUCTION

1.1 Fundamentals of Web-Based Sign Language Translation Platforms

A real-time sign language guidance and translation system evaluates visual hand configurations from camera feeds to recognize linguistic categories and synthesize them into verbal and textual speech. EchoSign combines deep convolutional feature extraction with an in-browser Region-of-Interest (ROI) canvas and a client-side speech vocalization engine. The fundamentals behind such a system are as follows:

1. **Digital Accessibility Gap:** Deaf and hard-of-hearing individuals encounter persistent barriers during spontaneous public interactions. Commercial translation services require human interpreters with high scheduling delays. A dedicated web portal serves as an immediate, private translation bridge.
2. **The Preprocessing Domain Gap:** Computer vision models frequently suffer severe accuracy drops when deployed to production. Standard training pipelines resize images using OpenCV area interpolation, while web applications decode base64 payloads using Pillow bilinear resampling. Preserving mathematical parity is essential for high-accuracy recognition.
3. **The Latency Dilemma:** Interactive communication tools require immediate responsiveness under strict sub-50 millisecond service level agreements (SLAs). Heavyweight neural networks running on server GPUs introduce cost and latency. The architecture must utilize quantized lightweight models executable on consumer CPU threads.
4. **Hands-On Verification and Feedback:** Static letter classification must be paired with temporal debouncing. EchoSign integrates a 2-frame stability filter directly into the browser, ensuring deliberate character entry while filtering transitional finger motion.
5. **Scalability and High Availability:** The web service utilizes a stateless REST API layer hosted on serverless cloud infrastructure, guaranteeing zero cold-start crashes and uninterrupted access across diverse mobile and desktop devices.

1.2 Background Theory

The system rests on principles from convolutional computer vision, discrete tensor downsampling, aspect ratio preservation, and client-side speech synthesis state machines.

- **Aspect Ratio Geometry:** Non-uniform scaling of widescreen webcam frames distorts finger aspect ratios. Center-square cropping extracts min(W, H) from the frame, preserving natural anatomical angles.
- **Convolutional Invariance:** Sequential 3x3 convolution banks extract spatial hierarchical features, while max-pooling layers provide translation invariance against minor hand shifts.
- **Quantization Theory:** Converting 32-bit floating-point weights to IEEE 754 16-bit half-precision preserves the dynamic range required by convolutional filters while reducing model size by 84%, fitting within strict cloud execution ceilings.

1.3 Core Principles

- **Decoupled Ingestion & Persistence:** Frame capture is isolated from server storage, ensuring complete user privacy with zero frame logging.

- **Real-Time O(1) Lexical Lookup:** Predicted category indices map instantly to English meanings and descriptions using in-memory hash maps.
- **Preprocessing Parity by Design:** Bilinear interpolation and center-square cropping are identical across training and production.
- **Zero Peripheral Cost:** Standard consumer webcams replace expensive proprietary flex-sensor data gloves.

1.4 Problems in the Current System

- **Intrusive Sensor Burden:** Traditional glove devices require physical wiring, regular battery charging, and frequent recalibration.
- **High Accuracy Degradation:** Existing academic prototypes trained on clean datasets fail under webcam lighting shifts and aspect distortion.
- **Latency & Package Bloat:** Bulky deep models exceed serverless container size limits, leading to execution timeouts.

CHAPTER 2
LITERATURE SURVEY

2.1 Overview of Existing Systems

Sign language translation has evolved over three decades from electromechanical sensor gloves to computer vision classifiers. Most existing solutions address isolated symptoms: either wearable hardware without web accessibility, or desktop Python notebooks without end-to-end user interfaces or speech synthesis.

2.2 Literature Review

2. 2.1 Technique 1: Wearable Sensor-Instrumented Data Gloves

Reference: CyberGlove & SmartGlove Systems (IEEE Sensors Journal, 2018). Method: Resistive bend sensors on digits with wrist IMUs. Functionality: High-precision joint angle tracking. Limitations: Cost-prohibitive, physically cumbersome, and socially stigmatizing for everyday use.

2. 2.2 Technique 2: Isolated Classical Computer Vision Contours

Reference: Color Space Segmentation & Hu Moments (Pattern Recognition Letters, 2017). Method: YCbCr skin-tone thresholding and morphological filtering. Functionality: Fast execution on low-power hardware. Limitations: Catastrophically brittle under varied illumination, background skin-colored clutter, and dark skin tones.

2. 2.3 Technique 3: Coordinate Landmark Regressors (MediaPipe / OpenPose)

Reference: Google MediaPipe Hands (CVPR, 2020). Method: Palm detection followed by 21 3D knuckle regression. Functionality: Extremely low memory footprint. Limitations: Struggles during self-occluding hand shapes ('E', 'M', 'N', 'S', 'T') where finger joints overlap closed fists.

2. 2.4 Technique 4: Deep Convolutional Neural Networks (CNNs)

Reference: Spatial Representation Learning via CNNs (IEEE Access, 2021). Method: Multi-layer 2D convolutions directly on pixel rasters. Functionality: Learns illumination-invariant textural and contour features. Limitations: High parameter counts require dedicated GPUs unless quantized.

2.3 Comparative Summary

Approach	Key Advantages	Primary Technical Bottlenecks	Inference Latency	Accuracy
Sensor Gloves	Hardware joint tracking	Costly ($500+), physically cumbersome	10–20 ms	92–95%
Classical Vision	No physical sensors	Fails under background clutter	60–120 ms	65–75%
MediaPipe + MLP	Lightweight tracking	Fails on finger self-occlusions	30–50 ms	85–90%
EchoSign (Proposed)	Pillow Parity + TFLite Float16 + Speech	Requires ambient light visibility	16.1 ms (median)	96.75%

Table 2.1: Comparative Summary of Existing Gesture Portals vs. EchoSign

CHAPTER 3
INFERENCES FROM LITERATURE

3.1 Inferences Drawn from Literature Survey

Inference from Technique 1 (Sensor Gloves): Physical sensing is unacceptable for spontaneous daily interactions. Systems must rely on optical capture over standard webcams.

Inference from Technique 2 (Classical Vision): Handcrafted feature descriptors break down in uncontrolled lighting. End-to-end deep learning is mandatory for robust edge extraction.

Inference from Technique 3 & 4 (Deep CNNs vs Landmarks): Landmark regressors struggle with overlapping closed fists. Full-frame CNNs preserve critical shadow and skin texture cues needed to distinguish letters like M, N, and T.

Summary of Observations: An accessible platform requires guaranteed preprocessing parity, aggressive model quantization, client-side temporal debouncing, and direct voice synthesis in a single unified web portal.

CHAPTER 4
PROBLEM STATEMENT AND SOLUTION APPROACH

4.1 Problem Statement

Deaf and hard-of-hearing individuals navigate acute communication barriers during daily social, healthcare, and civic encounters. Existing technological solutions either require costly wearable data gloves or provide isolated Python scripts completely detached from responsive user interfaces. Furthermore, common image downsampling discrepancies between training scripts and production web servers severely degrade classification accuracy on live webcam feeds, causing misclassifications and user frustration.

4.2 Proposed Solution Approach

EchoSign is an end-to-end, decoupled three-tier web application combining client-side canvas stream rendering with a lightweight Flask RESTful API, a float16-quantized TensorFlow Lite inference engine, and native browser speech vocalization.

Core Subsystem Workflows:

1. **Ingestion & Preprocessing:** WebRTC video feeds are cropped to a central 70% ROI canvas, converted to base64, cropped to square aspect, and resized via Pillow bilinear interpolation down to 64x64 pixels.
2. **Decision and Scoring:** The double-convolution deep neural network outputs a 29-class softmax probability distribution. Predictions with confidence >= 50% are validated against a 2-frame stability debounce.
3. **Sentence Composition:** Verified alphabet characters are appended to an editable text buffer, while 'del' triggers backspace deletion and 'space' introduces word separators.
4. **Speech Synthesis:** The native Web Speech API (window.speechSynthesis) vocalizes the finalized text buffer into spoken English audio.

4.3 Methodology

- **Phase 1:** Problem Formulation & Needs Analysis: Literature review and identification of gaps in sign language communication aids.
- **Phase 2:** Requirement Engineering & SLA Definition: Setting formal SLAs (sub-50 ms inference latency, cross-browser desktop/mobile compatibility).
- **Phase 3:** Dataset Curation & Partitioning: Compiling a balanced 20,300-image corpus across 29 categories from the ASL Alphabet dataset.
- **Phase 4:** Preprocessing Parity Engineering: Implementing identical center-square cropping and Pillow bilinear downsampling in training and API loaders.
- **Phase 5:** Deep CNN Architecture & Training: Designing double-conv blocks, training for 14 epochs with Adam and learning rate decay, achieving 96.75% accuracy.
- **Phase 6:** Model Quantization & Cloud Deployment: Compressing weights to float16 TFLite (4.29 MB) and deploying to Vercel serverless edge infrastructure.
- **Phase 7:** Software Testing & Verification: Executing Pytest automated test suites, measuring latency, and validating confidence thresholds.

4.4 Performance Metrics & Threshold Optimization

Classification Accuracy: EchoSign achieves an overall test accuracy of 96.75% across 2,030 held-out test images, correctly classifying 28 out of 29 classes with precision exceeding 92%.

Confidence Threshold Optimization: Table 4.1 documents system behavior across candidate acceptance thresholds.

Threshold (t)	Recognition Recall	False Entry Count	Operational Assessment
t = 0.30 (Permissive)	98.8%	142	High false positive character insertions
t = 0.85 (Overly Strict)	74.2%	8	Excludes valid hand configurations
t* = 0.50 (Optimal)	96.75%	14	Optimal Balance (High Precision & Low Noise)

Table 4.1: Performance across Candidate Classification Thresholds

CHAPTER 5
SYSTEM ARCHITECTURE & IMPLEMENTATION DETAILS

5.1 Data Flow Diagrams & Subsystem Architecture

The system behavior is formally represented through hierarchical Data Flow Diagrams (DFD) illustrating core data transformations between camera ingestion, serverless preprocessing, tensor inference, and client vocalization.

Subsystem Cardinality Mapping: The architecture defines a 1:1 relationship between an ingested base64 image and a preprocessed 64x64x3 tensor, and a 1:N mapping between a user's signing session and consecutive assembled sentence tokens.

DFD Functional Subsystems:

- **1.0 Camera & ROI Subsystem:** Captures 70% centered video frames, serializes pixels to base64 JPEG strings.
- **2.0 Ingestion & Preprocessing Service:** Validates MIME types, extracts center-square aspect crop min(W, H), and resamples via Pillow bilinear interpolation.
- **3.0 Deep Inference Engine:** Loads float16 TFLite model via `ai-edge-litert`, executes forward tensor pass, generating 29-class softmax vectors.
- **4.0 Lexical & Speech Synthesizer:** Resolves top prediction, applies 2-frame debounce stability filter, appends character to sentence buffer, and synthesizes speech via Web Speech API.

5.2 Preprocessing Schemas & REST API Specifications

Pipeline Stage	Data Structure	Data Type	Constraints & Validation Rules
Raw Payload	Base64 String	String	MIME prefix check, max 10 MB payload size
Cropped Image	Square PIL Image	PIL.Image (RGB)	min(W, H) square extraction, center aligned
Resampled Image	64x64 Grid	PIL.Image	Resampling.BILINEAR interpolation
Input Tensor	Normalized Array	Float32 Array	Shape: (1, 64, 64, 3), range [0.0, 1.0]
Output Probabilities	Softmax Vector	Float32 Array	29 Class probabilities summing to 1.0

Table 5.1: Pipeline Preprocessing Schemas, Types and Constraints

RESTful API Route Contracts: Table 5.2 defines the HTTP interface exposed by the Flask backend service.

Method	URI Route	Authorization	Description & Expected Response
GET	/api/health	Public	Returns JSON 200 with service health and

			model_loaded status
POST	/api/predict	Public	Ingests base64 frame, executes TFLite inference, returns label & confidence
POST	/api/predict-video	Public	Accepts multipart video file, extracts sample frames, returns sequence

Table 5.2: RESTful API Route Specifications and Contract Definitions

5.3 Deep CNN Architecture & Mathematical Formulation

1. Center-Square Aspect-Ratio Crop Formulation:

Given input frame I of width W and height H, let D = min(W, H). Spatial offsets are computed as:

```
x_start = floor((W - D) / 2),    y_start = floor((H - D) / 2)
I_crop = I[y_start : y_start + D,  x_start : x_start + D]
```

2. Bilinear Interpolation Resampling Formulation:

Continuous target grid coordinates are mapped to four discrete neighbor pixels (x1, y1), (x1, y2), (x2, y1), (x2, y2). The resampled color intensity f(x, y) is calculated as:

```
f(x, y) = (1 - alpha)(1 - beta) I(y1, x1) + alpha(1 - beta) I(y1, x2) + (1 -
alpha)beta I(y2, x1) + alpha beta I(y2, x2)
X = f(x, y) / 255.0, where X in [0.0, 1.0]^(1 x 64 x 64 x 3)
```

3. Convolutional Forward Propagation & Softmax Formulation:

```
Z^(l) = ReLU( W^(l) * A^(l-1) + b^(l) )
```

The output logits z in R^29 are normalized to categorical probabilities via Softmax:

```
p_i = exp(z_i) / sum(exp(z_j)),   for all i in {1, 2, ..., 29}
```

4. Temporal Debounce and Speech Synthesis:

To eliminate accidental double-typing from hand transitions, the client enforces a 2-frame stability check: character C is committed if Predicted(t) == Predicted(t-1) with Confidence >= 0.50. Upon user trigger, window.speechSynthesis.speak() instantiates an asynchronous SpeechSynthesisUtterance object to vocalize the finalized English sentence aloud.

5.4 Software Testing Matrix & Execution Verification

Automated testing was conducted across unit, preprocessing, API routing, and model layers using Pytest. Table 5.3 summarizes execution results for the 8 core automated test suites.

Test ID	Test Target	Input Payload Condition	Expected Result	Status
TC-01	Tensor Shape	Valid 200x200 RGB image	Tensor shape (1, 64, 64, 3)	PASSED
TC-02	Aspect Ratio	Widescreen 1280x720 frame	Center-square crop min(W, H)	PASSED
TC-03	Corrupt Input	Invalid byte payload	ValueError raised cleanly	PASSED
TC-04	Lexical Lookup	Query letter 'A'	Returns description & meaning	PASSED
TC-05	Health Endpoint	GET /api/health	HTTP 200 + model_loaded: true	PASSED
TC-06	Prediction API	POST /api/predict (base64)	HTTP 200 + label & confidence	PASSED
TC-07	Missing Payload	POST /api/predict (empty)	HTTP 400 Bad Request	PASSED

TC-08	TFLite Model	2,030 test samples	96.75% Test Accuracy	PASSED

Table 5.3: Automated Test Suite Verification Results (8/8 Passed)

Latency Accounting Across Architecture Tiers: Table 4.2 documents processing latencies measured across each pipeline stage against the 50 ms SLA.

Processing Stage	Median Latency	p95 Latency	Gateway SLA Margin
1. Base64 Network Ingestion	2.10 ms	4.50 ms	> 45 ms remaining
2. Aspect Ratio Center Crop	0.85 ms	1.40 ms	> 48 ms remaining
3. Bilinear Resampling & Norm	1.95 ms	3.20 ms	> 46 ms remaining
4. TFLite Tensor Forward Pass	11.20 ms	15.60 ms	> 34 ms remaining
Total Server Pipeline Execution	16.10 ms	24.70 ms	25.30 ms peak cushion

Table 4.2: Latency Accounting across Pipeline Stages vs the 50 ms SLA

CONCLUSION

EchoSign demonstrates that an integrated, decoupled web architecture can successfully bridge the communication divide between American Sign Language users and hearing individuals. The responsive client frontend paired with a lightweight Flask controller and an optimized float16 TensorFlow Lite model provides users with real-time fingerspelling recognition alongside immediate speech synthesis. By enforcing strict preprocessing parity through center-square cropping and Pillow bilinear resampling, the system achieves an outstanding 96.75% classification accuracy across 2,030 test images with a median response latency of 16.1 ms, well within the 50 ms service level agreement. Stateless cloud serverless deployment and browser-native voice vocalization guarantee zero cost and robust accessibility, establishing a practical assistive communication platform.

REFERENCES

[1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.

[2] Akash Nagaraj. (2018). ASL Alphabet Dataset. Kaggle Datasets, DOI: 10.34740/KAGGLE/DSV/26430.

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press, Cambridge, MA.

[4] David, R., et al. (2021). TensorFlow Lite: On-device machine learning framework. IEEE Micro, 41(4), 106–114.

[5] Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.

[6] Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition: a survey. AI Review, 43(1), 1–54.

[7] Bantupalli, K., & Xie, Y. (2018). ASL recognition using deep learning. IEEE Big Data, pp. 4396–4399.

[8] W3C Speech API Community Group. (2020). Web Speech API Specification. World Wide Web Consortium.

[9] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.

[10] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

[11] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

[12] IEEE Standard Association. (2017). IEEE Standard for Software and System Test Documentation. IEEE Std 829-2017.